

Centro Universitário da Fundação Hermínio Ometto - FHO
Engenharia da Computação - 6º semestre
Projeto Integrador

David Alves Molizane Neto - RA 116195
Eric Aparecido Savassi - RA 116822
Luis Felipe Travagim Braz Pinto - RA 116508
Joseph Costa Anjos - RA 115767
Pedro Antônio Rodrigues - RA 117776

Tarefa 3 - Mapa Funcional e Fluxo de Dados

DataPlanner: organização funcional da plataforma e plano de implementação

Arara, SP
Setembro de 2026

1 O problema que o DataPlanner ajuda a resolver

O DataPlanner é uma plataforma de apoio à decisão: ela guarda informações, recupera o que é relevante, compara alternativas e apresenta um resultado que ajude alguém a decidir melhor. Para dar corpo a essa ideia, o grupo escolheu como cenário de referência o planejamento de entregas de uma transportadora de pequeno porte.

Hoje esse tipo de operação costuma ser organizada em planilhas ou no critério pessoal de quem monta as rotas, o que gera atraso em entregas urgentes e veículos sendo mal aproveitados. A decisão que o sistema precisa apoiar é simples de enunciar e difícil de fazer bem à mão: quais entregas devem sair primeiro e em qual veículo, considerando prazo, capacidade e distância. O DataPlanner não decide isso sozinho, ele organiza os dados e sugere uma ordem, deixando a palavra final com quem opera o sistema.

2 Como dividimos as funções do sistema

Separamos o DataPlanner em cinco módulos, cada um com uma responsabilidade única. O armazenamento de dados não é um módulo de decisão; ele existe como apoio, guardando o que os outros módulos produzem e consomem.

Quadro 1 - Módulos do DataPlanner aplicados ao cenário de entregas

Módulo	Entrada	Ação	Saída
Cadastro	Clientes, entregas, veículos, prazos e prioridades.	Registrar e manter esses dados atualizados.	Dados disponíveis para os demais módulos.
Consulta	Dados armazenados e filtro escolhido pelo usuário.	Localizar entregas pendentes por região ou urgência.	Lista filtrada de entregas.
Análise	Entregas filtradas e veículos disponíveis.	Comparar prazo, capacidade, distância e custo.	Indicadores e alternativas possíveis.
Otimização	Alternativas, indicadores e restrições (ex.: capacidade máxima).	Ordenar e priorizar as alternativas segundo os critérios definidos.	Lista de alternativas priorizadas.
Apresentação	Alternativas priorizadas e indicadores.	Montar uma visão clara para quem vai decidir.	Lista final, indicadores e alertas na tela.

Fonte: elaboração do grupo.

Vale registrar uma distinção que usamos para pensar no sistema: entrega, veículo e rota são conceitos específicos do nosso cenário, mas cadastrar, consultar, comparar, priorizar e apresentar são funções genéricas, se um dia o DataPlanner

precisasse organizar chamados de suporte em vez de entregas, esses cinco módulos continuariam fazendo sentido, só mudaria o que circula dentro deles.

3 Fluxo das informações

O diagrama abaixo mostra o caminho que um dado percorre dentro do sistema. Cada seta representa o que passa de um módulo para o outro.

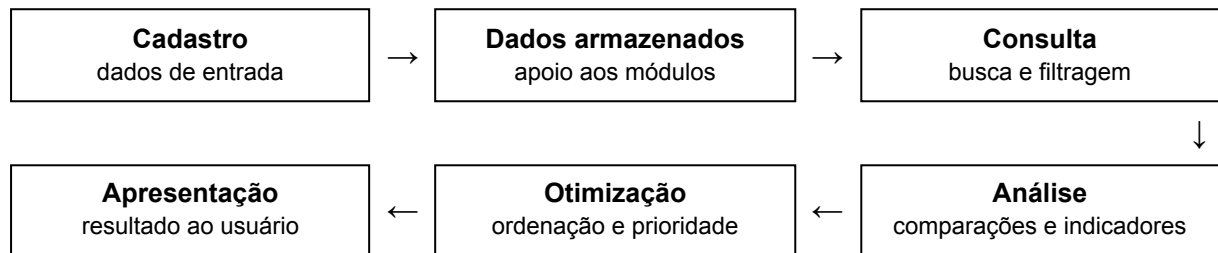


Figura 1 - Fluxo funcional do DataPlanner. Fonte: elaboração do grupo.

3.1 Acompanhando um caso do começo ao fim

Um exemplo ajuda a enxergar o fluxo funcionando na prática. O operador cadastra uma entrega urgente e os veículos disponíveis naquele dia, e esses dados ficam guardados e prontos para uso. Em seguida ele consulta quais entregas urgentes ainda estão pendentes; o sistema compara prazo, distância e capacidade de cada veículo disponível e monta as alternativas possíveis. Essas alternativas são ordenadas por prioridade e viabilidade, e o resultado chega ao operador já acompanhado dos indicadores que explicam por que aquela ordem foi sugerida, a decisão final continua sendo dele.

4 Ferramentas que pretendemos usar e as dificuldades que já enxergamos

Esta parte vai além do mapa funcional: é o nosso planejamento de como pretendemos construir o que descrevemos acima, e de onde já conseguimos prever atrito. As disciplinas de Banco de Dados e Estruturas de Dados, que estamos cursando agora, guiaram boa parte dessas escolhas.

A ideia é manter os dados em um banco relacional (pensamos em PostgreSQL), porque clientes, entregas e veículos se relacionam de um jeito que um banco relacional representa bem, sem duplicar informação. O cadastro seria um formulário web simples gravando direto nessas tabelas, e a consulta usaria SQL com índices nas colunas mais filtradas, como região, status e prioridade, já que buscar é a ação que o operador mais vai repetir no dia a dia.

4.1 O ponto que mais nos preocupa: fazer o Python conversar com o banco

O módulo de Análise é onde prevemos a maior dificuldade real de implementação. A conta e a comparação dos indicadores pretendemos fazer em Python, mas os dados moram no PostgreSQL, ou seja, cada análise depende do Python abrir uma conexão com o banco (via algo como psycopg2 ou SQLAlchemy), buscar o que precisa e só então processar. Isso traz decisões que ainda não tínhamos parado para pensar direito: filtrar e agregar já dentro do SQL (mais rápido, mas exige escrever consultas mais elaboradas) ou trazer os dados brutos para o Python e processar com as estruturas que estamos aprendendo em Estruturas de Dados (mais simples de programar, mas com risco de puxar volume desnecessário e pesar a memória à toa).

Tem também um problema mais chato do que parece: tipos de dado não chegam do banco exatamente como o Python espera, datas, valores decimais e nulos já causaram confusão em exercícios menores da disciplina, e numa análise com várias tabelas envolvidas esse tipo de erro fica mais difícil de rastrear. Como Banco de Dados e Estruturas de Dados são matérias que estamos vendo ao mesmo tempo, sabemos que essa ponte entre os dois lados vai exigir um tempo real de ajuste antes da análise funcionar de forma confiável, e não só de "ligar as peças".

Depois que os dados voltam tratados do banco, a ideia para o módulo de Otimização é usar uma fila de prioridade (um heap) para ordenar as alternativas, e para a Apresentação, uma interface web simples em HTML/CSS, sem preocupação estética, só clareza para o operador decidir rápido. Para organizar o trabalho entre as cinco pessoas do grupo, pretendemos usar Git/GitHub para o código e um quadro Kanban (Trello ou GitHub Projects) para dividir tarefas.

Nada disso está implementado ainda, e a dificuldade do Python com o banco é uma previsão nossa, não algo que já testamos na prática, é o plano que vamos seguir e ajustar nas próximas etapas, conforme formos avançando em Banco de Dados e Estruturas de Dados.

5 Revisando a organização que propusemos

Usamos quatro perguntas para conferir se a divisão em módulos faz sentido, e essa revisão nos deixou confortáveis com o resultado:

- **Coesão:** cada módulo trata de um assunto só, cadastro não filtra, consulta não calcula indicador, otimização não decide sozinha.
- **Acoplamento:** baixo. Cada módulo recebe só o que precisa para trabalhar, a otimização, por exemplo, não precisa saber o nome do cliente, só prazo, capacidade e prioridade.

- **Integração:** está clara. O Quadro 1 mostra exatamente o que cada seta do fluxograma carrega de um módulo para o próximo.
- **Caráter genérico:** os módulos sobrevivem a uma troca de domínio; só as entidades específicas (entrega, veículo) mudariam.

Com isso, também conferimos os pontos exigidos nesta etapa: cada função aparece em um único módulo; está claro o que fica armazenado e o que passa de módulo para módulo; é possível seguir um dado desde o cadastro até o resultado final; e busca, ordenação e priorização estão identificadas como necessidades futuras, sem implementação nesta etapa.

6 Considerações finais

Este documento fecha a etapa conceitual do DataPlanner: definimos os cinco módulos, o caminho que a informação percorre entre eles e um caso completo usando entregas como exemplo. A partir daqui, o próximo passo natural é usar Banco de Dados para desenhar as tabelas e relacionamentos por trás do módulo de cadastro, e Estruturas de Dados para implementar de fato a busca, a ordenação e a priorização que hoje existem só no papel.